ICM Module Analysis Report

Learned ICMNet vs Exact Malmuth-Harville ICM

Training run: Post-bugfix 6.3M tournaments (20260416_155300_916856)

Agents: 4 independent agents (no weight sharing)

Checkpoint: Final checkpoint at 6,300,000 tournaments

Date: 2026-04-19

Script: analysis/icm_compare.py

1. Background

1.1 What is ICM?

The **Independent Chip Model (ICM)** is the standard method for converting tournament chip stacks into prize equity. The **Malmuth-Harville** variant recursively computes the probability that each player finishes in each position (proportional to chip share), then weights those probabilities by the prize pool.

For a 4-player sit-and-go with prize pool [+1.5, +0.5, -0.5, -1.5]:

• A player with more chips has higher equity (monotonic).
• Each additional chip is worth less than the previous (diminishing returns / concavity).
• Equal stacks produce equal equities.

1.2 Architecture

The Poker_DQN model has two neural-network components:

```
ICMNet: Linear(8, 128) → ReLU → Linear(128, 4) → Softmax
  Input: 4 normalized stacks + 4-dim prize pool = 8
  Output: 4-dim probability simplex
```

```
DuelingDQN: Linear(12, 128) → ReLU → Linear(128, 128) → ReLU
  → Value head(1) + Advantage head(2)
  Input: 8-dim base state + 4-dim ICM output = 12
  Output: Q(all-in), Q(fold)
```

The ICMNet is trained **indirectly**: there is no supervised ICM loss. Gradients flow from the DQN's TD-error loss backward through the concatenated state into the ICMNet weights. The question is: **what equity function did the ICMNet learn, and how does it compare to exact ICM?**

2. Methodology

Four analyses were performed on the final checkpoint (6.3M tournaments):

Analysis	What it measures
Stack sweep	How each function responds as one player's stack varies (1-95 BB), others fixed at 25BB
Scatter plot	Relationship between exact ICM and learned ICM across 100 random stack distributions
Decision impact	Push/fold decisions when same DQN is fed learned ICM output vs exact ICM output
Property comparison	Monotonicity, rank ordering agreement, and key scenario outputs

All evaluations use 100 random stack distributions generated via Dirichlet sampling (total 200 chips, minimum 2 chips per player, seed=42 for reproducibility).

3. Results

3.1 Stack Sweep

Player 0's stack is swept from 1BB to 95BB while the other 3 players are fixed at 25BB. The plot shows the learned ICMNet softmax output (colored, left axis) against exact ICM raw equity and normalized equity (black, right axis).

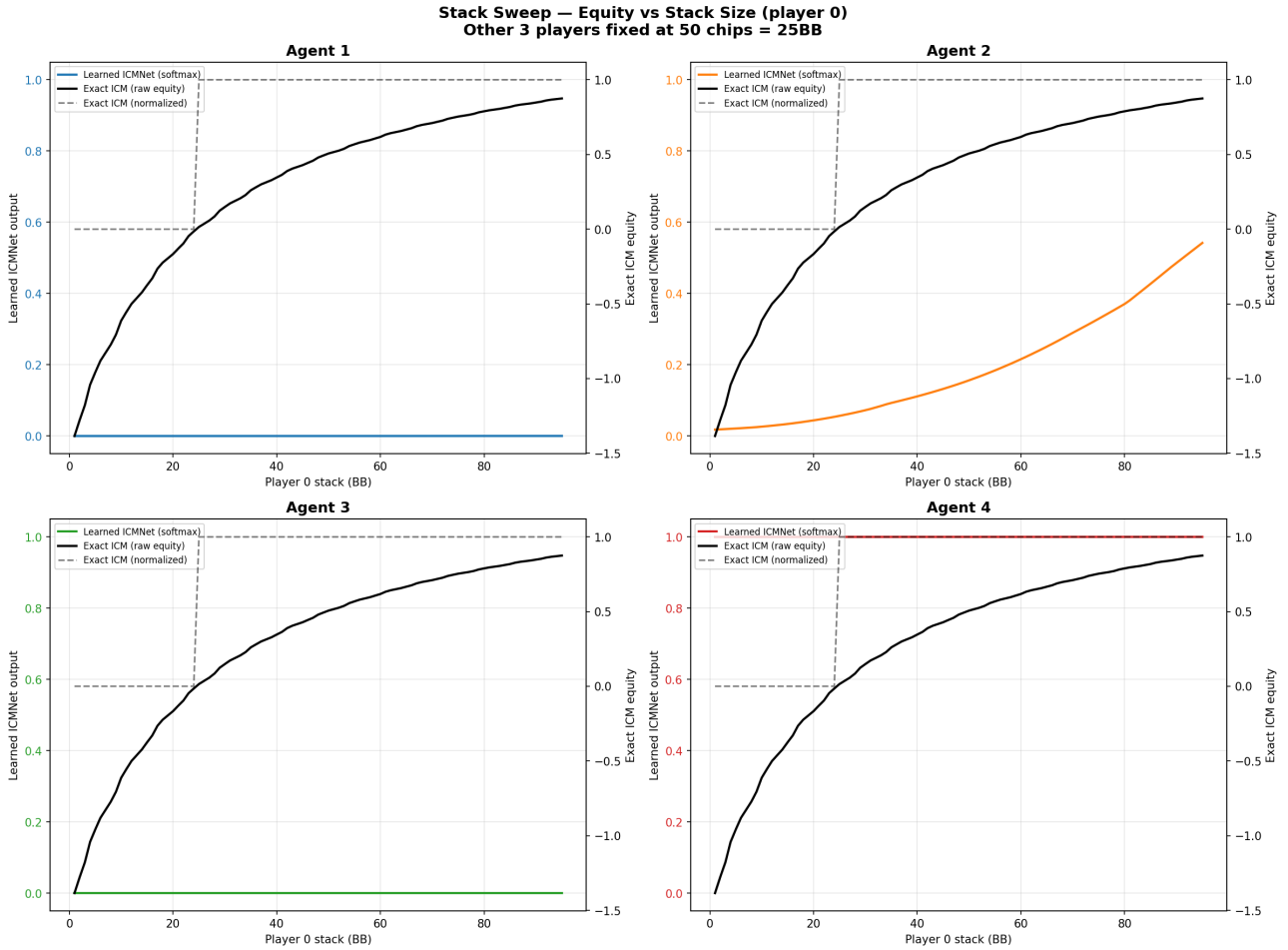

Figure 1: Stack sweep — equity vs stack size.

Exact ICM shows the expected concave monotonic curve: equity rises from -1.5 (last-place) to +1.5 (first-place) with diminishing returns per additional chip. This is the classic ICM pressure effect.

Learned ICMNet outputs near-constant values regardless of stack size for all 4 agents:

Agent	Learned output for player 0	Interpretation
Agent 1	~0.00 (flat)	All mass on player index 3
Agent 2	~0.03-0.07 (flat)	All mass on player index 2
Agent 3	~0.00 (flat)	All mass on player index 2
Agent 4	~1.00 (flat)	All mass on player index 0

The learned ICMNet shows **no sensitivity to stack sizes**. It has collapsed to a near-constant one-hot vector that does not vary as stacks change.

3.2 Scatter Plot

For 100 random stack distributions, both exact ICM (normalized) and learned ICMNet output are computed for all 4 players. Each dot represents one player in one distribution (400 dots per agent). A perfect match would place all points on the diagonal.

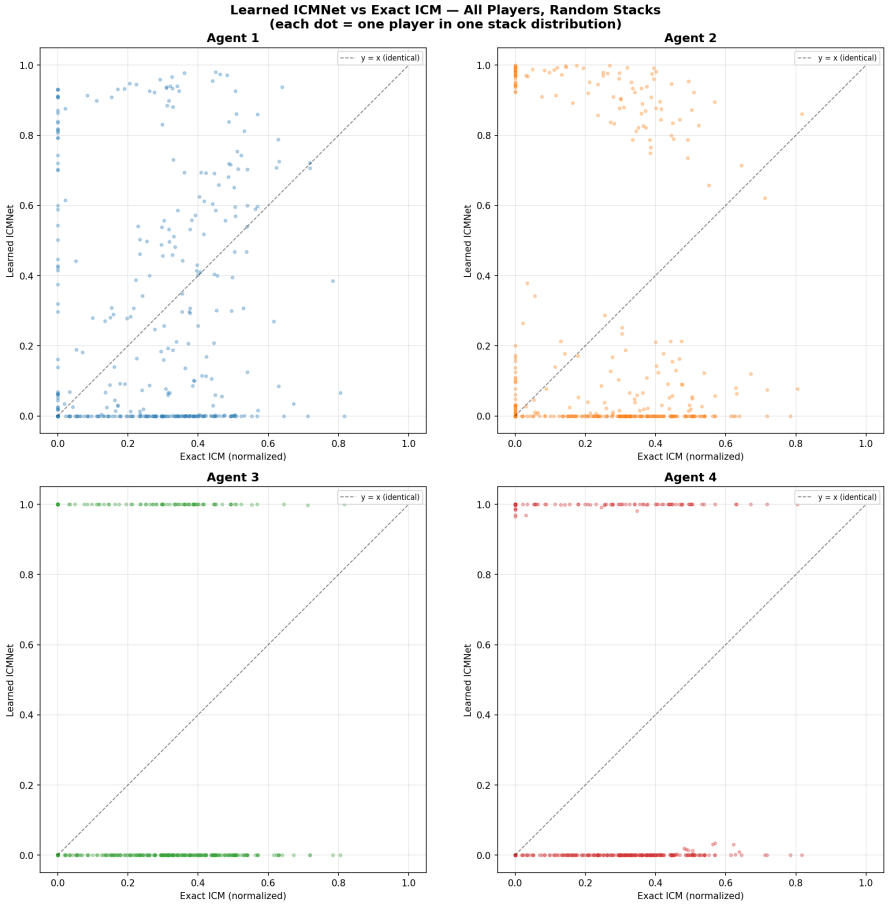

Figure 2: Learned ICMNet vs exact ICM scatter.

Points cluster at y≈0 and y≈1 rather than along the diagonal, confirming the ICMNet outputs near-binary values with no correlation to exact ICM equity. Agent 1 shows the most spread but still no meaningful relationship.

3.3 Decision Impact

For each agent, the 13×13 push/fold range card is computed twice: once using the learned ICMNet output, once replacing it with exact ICM (normalized to simplex). Both use greedy action selection, position-averaged across all 4 positions, at 25BB stack depth.

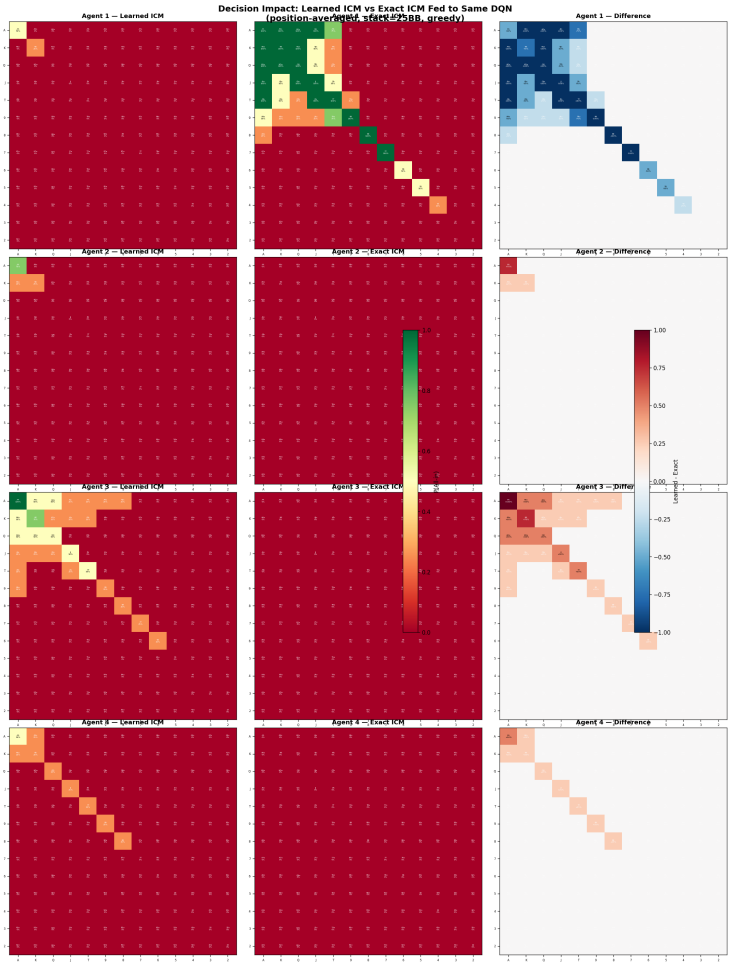

Figure 3: Decision impact — learned ICM (left), exact ICM (center), difference (right).

Agent	Push agreement	Mean |diff|	Interpretation
Agent 1	148/169 (87.6%)	0.154	Most divergence; exact ICM widens push range on high cards
Agent 2	168/169 (99.4%)	0.007	Near-identical decisions
Agent 3	167/169 (98.8%)	0.059	Near-identical decisions
Agent 4	169/169 (100.0%)	0.015	Perfectly identical decisions

Key finding: Despite the ICMNet learning nothing resembling actual ICM equity, **87.6% to 100% of push/fold decisions remain the same** when swapping in exact ICM. The DQN has learned to largely **ignore the ICM features** and make decisions based on the 8-dim base state (hand strength, stack, position, etc.).

3.4 Property Comparison

Monotonicity (bigger stack → higher equity):

	Exact ICM	Agent 1	Agent 2	Agent 3	Agent 4
Monotonicity	100.0%	53.3%	49.7%	48.2%	49.2%

Exact ICM is perfectly monotonic. All learned ICMNets are ~50%, equivalent to random chance (a coin flip predicts “bigger stack = higher equity” 50% of the time).

Rank ordering agreement:

Agent 1	Agent 2	Agent 3	Agent 4
5.0%	6.0%	1.0%	2.0%

Random chance produces the correct ranking of 4 players 1/24 = 4.2% of the time. All agents are near this baseline.

Spearman rank correlation:

Agent 1	Agent 2	Agent 3	Agent 4
0.098	-0.022	-0.026	-0.026

All near zero — no rank correlation.

Key scenario outputs (final checkpoint):

Agent 1:

Scenario	Exact (norm)	Learned
Equal [50 50 50 50]	[0.250 0.250 0.250 0.250]	[0.000 0.116 0.000 0.884]
Desc. [80 60 40 20]	[0.438 0.347 0.215 0.000]	[0.000 0.130 0.000 0.870]
Dominant [140 20 20 20]	[1.000 0.000 0.000 0.000]	[0.512 0.009 0.000 0.479]
Short [66 66 66 2]	[0.333 0.333 0.333 0.000]	[0.000 0.232 0.000 0.768]

Agent 2:

Scenario	Exact (norm)	Learned
Equal [50 50 50 50]	[0.250 0.250 0.250 0.250]	[0.057 0.000 0.943 0.000]
Desc. [80 60 40 20]	[0.438 0.347 0.215 0.000]	[0.035 0.000 0.965 0.000]
Dominant [140 20 20 20]	[1.000 0.000 0.000 0.000]	[0.089 0.000 0.911 0.000]
Short [66 66 66 2]	[0.333 0.333 0.333 0.000]	[0.046 0.000 0.954 0.000]

Agent 3:

Scenario	Exact (norm)	Learned
Equal [50 50 50 50]	[0.250 0.250 0.250 0.250]	[0.000 0.000 1.000 0.000]
Desc. [80 60 40 20]	[0.438 0.347 0.215 0.000]	[0.000 0.000 1.000 0.000]
Dominant [140 20 20 20]	[1.000 0.000 0.000 0.000]	[0.000 0.000 1.000 0.000]
Short [66 66 66 2]	[0.333 0.333 0.333 0.000]	[0.000 0.000 1.000 0.000]

Agent 4:

Scenario	Exact (norm)	Learned
Equal [50 50 50 50]	[0.250 0.250 0.250 0.250]	[1.000 0.000 0.000 0.000]
Desc. [80 60 40 20]	[0.438 0.347 0.215 0.000]	[1.000 0.000 0.000 0.000]
Dominant [140 20 20 20]	[1.000 0.000 0.000 0.000]	[1.000 0.000 0.000 0.000]
Short [66 66 66 2]	[0.333 0.333 0.333 0.000]	[1.000 0.000 0.000 0.000]

4. Analysis

4.1 Why did the ICMNet collapse?

The ICMNet is trained via **indirect gradient flow**: the only learning signal comes from the DQN's TD-error loss, which backpropagates through the concatenated 12-dim state into the ICMNet weights. There is no direct supervision telling the ICMNet what “correct” equities look like.

Several factors contribute to the collapse:

1. No supervised loss. Without an explicit ICM target, the ICMNet only receives gradients that improve Q-value prediction. If the DQN can predict Q-values adequately from the 8-dim base state alone, the ICM gradients become uninformative noise.

2. Softmax saturation. The softmax output forces 4 outputs to sum to 1. Once one logit dominates, the softmax saturates and gradients for all outputs shrink exponentially (vanishing gradient through softmax). This creates a stable equilibrium where the collapsed one-hot output persists.

3. DQN adaptation. Even if the ICMNet produced useful information early in training, the DQN adapts to whatever the ICMNet currently outputs. Once collapse begins, the DQN compensates by relying more on the base state, further reducing the gradient signal to the ICMNet.

4. Symmetry breaking. Each agent's ICMNet independently collapses to a different one-hot vector (Agents 2-3 to index 2, Agent 4 to index 0, Agent 1 mixed). The initial random weights determine which index “wins” the softmax competition.

4.2 What does the DQN actually use?

The decision impact analysis shows near-identical push/fold decisions whether the DQN receives learned or exact ICM values. This confirms:

• The **8-dim base state** (hand ranks, suited flag, stack in BB, active players, shortest stack flag, position, call/pot ratio) carries essentially all decision-relevant information.
• The **4-dim ICM output** has been absorbed into the DQN's bias terms as a constant offset.
• The DQN's own `stack_norm` feature already captures much of what ICM would provide for push/fold decisions at a given stack depth.

4.3 Is the learned ICM better or worse than exact ICM?

Neither. The learned ICMNet has not learned any equity function at all. It outputs a constant vector that carries no information about relative chip positions. Exact ICM correctly captures monotonicity, diminishing returns, and stack ordering. The learned ICMNet captures none of these.

However, this does not mean the overall model performs poorly. The DQN has learned reasonable push/fold decisions using the base state features alone. The ICMNet simply does not contribute.

5. Implications and Recommendations

The ICMNet module is **non-functional** in its current form. It acts as a constant bias vector that the DQN has learned to work around. This is an architectural issue, not a training duration issue: the collapse happens early and is self-reinforcing.

Potential fixes:

1. **Supervised pre-training.** Pre-train the ICMNet against exact Malmuth-Harville targets before starting RL. Then either freeze it or fine-tune with a small learning rate alongside the DQN.

2. **Auxiliary ICM loss.** Add a supervised loss term alongside the TD loss: L = L_TD + λ · L_ICM, where L_ICM compares ICMNet output against exact ICM values computed from observed stacks.

3. **Remove the softmax.** Replace the softmax output with raw linear or ReLU outputs that don't have the saturation problem. The DQN can learn to interpret unnormalized values.

4. **Direct ICM injection.** Skip the learned network entirely and feed exact ICM values as features. This guarantees correct equity information.

5. **Remove ICMNet entirely.** If the DQN performs adequately with the 8-dim base state, the ICMNet adds complexity without benefit. The simpler DQNAgent class already exists as an alternative.

Reproducibility

```
cd pokerDQN
python analysis/icm_compare.py --output-dir results/new_run/icm
```

Fixed random seed (42) for stack generation. All evaluations use the final checkpoint at 6,300,000 tournaments from run 20260416_155300_916856.